

Rachunek Macierzowy

Projekt 4

Natalia Bratek, Joanna Konieczny

Treść zadania

W ramach czwartego projektu napisano w języku *Python* program, który dla macierzy prostokątnej A o wymiarach $n \times m$ oblicza SVD za pomocą następujących kroków:

1. Wypisanie/narysowanie macierzy A .
2. Obliczenie i wypisanie macierzy $A \cdot A^T$ (wymiar $n \times n$).
3. Obliczenie wartości λ_i i wektorów własnych U_i macierzy $A \cdot A^T$.
4. Wypisanie/narysowanie macierzy wektorów własnych $[U_1, U_2, \dots, U_n]$ oraz macierzy diagonalnej S takiej, że $S_{ii} = \sqrt{\lambda_i}$.
5. Obliczenie macierzy V korzystając z własności $V = A^T \cdot U \cdot S^{-1}$.
6. Wypisanie/narysowanie macierzy $[V_1, V_2, \dots, V_m]^T$.
7. Obliczenie i wypisanie macierzy $A^T \cdot A$ (wymiar $m \times m$).
8. Obliczenie wartości λ_i i wektorów własnych V_i macierzy $A^T \cdot A$.
9. Wypisanie/narysowanie macierzy $[V_1, V_2, \dots, V_m]^T$ i obliczenie i wypisanie macierzy diagonalnej S takiej, że $S_{ii} = \sqrt{\lambda_i}$.
10. Obliczenie macierzy U korzystając z własności $U = A \cdot V \cdot S^{-1}$.
11. Wypisanie/narysowanie macierzy $[U_1, U_2, \dots, U_n]$.
12. Porównanie dekompozycji.
13. Wyliczenie $\dim R(A)$ oraz $\dim N(A)$, gdzie:
 - $R(A)$ to obraz operatora A ,
 - $N(A)$ to jądro operatora A .

Do wykonania programu wykorzystano bibliotekę *numpy*.

Wstęp teoretyczny

Rozkład SVD (Singular Value Decomposition) jest jedną z fundamentalnych dekompozycji macierzy w algebrze liniowej. Każdą macierz rzeczywistą A o wymiarach $n \times m$ można przedstawić w postaci:

$$A = U \cdot S \cdot V^T$$

gdzie:

- U jest macierzą ortogonalną ($n \times n$), której kolumny są wektorami własnymi macierzy $A \cdot A^T$,
- V jest macierzą ortogonalną ($m \times m$), której kolumny są wektorami własnymi macierzy $A^T \cdot A$,
- S jest macierzą diagonalną zawierającą wartości singularne σ_i .

Wartości singularne są pierwiastkami z wartości własnych macierzy $A \cdot A^T$ lub $A^T \cdot A$:

$$\sigma_i = \sqrt{\lambda_i}$$

Liczba niezerowych wartości singularnych odpowiada rzędowi macierzy A , co pozwala wyznaczyć wymiar obrazu (R , range), odpowiadający liczbie liniowo niezależnych kolumn macierzy A oraz jądra (N , null space) macierzy, które określa liczbę niezależnych kierunków, które są redukowane przez odwzorowanie liniowe do zera.

Program i wyniki wywołania

W poniższych podsekcjach zaprezentowano fragmenty kodu odpowiadające kolejnym etapom instrukcji wraz z wynikiem ich wywołania.

1. Wypisanie/narysowanie macierzy A .

```
def generate_A(n, m):  
    A = [[random.randint(1, 10) for _ in range(m)] for _ in range(n)]  
    return np.array(A, dtype=float)  
  
A = generate_A(5, 6)  
n, m = A.shape  
  
print("1. A:\n", A)
```

Wynik

1. A:

```
[[ 1.  6.  1.  3.  1. 10.]  
 [ 6.  8.  7. 10.  5. 10.]  
 [ 7.  1.  8.  5.  2.  5.]  
 [ 8.  3.  8.  7.  7.  2.]  
 [ 1.  3. 10.  3.  5.  1.]]
```

2. Obliczenie i wypisanie macierzy $A \cdot A^T$ (wymiały $n \times n$).

```
AAT = A @ A.T  
print("\n2. A A^T:\n", AAT)
```

Wynik

2. A A^T:

```
[[148. 196.  88.  82.  53.]  
 [196. 374. 216. 253. 165.]  
 [ 88. 216. 168. 182. 120.]  
 [ 82. 253. 182. 239. 155.]  
 [ 53. 165. 120. 155. 145.]]
```

3. Obliczenie wartości λ_i i wektorów własnych U_i macierzy $A \cdot A^T$.

```
eigvals_U, U = np.linalg.eigh(AAT)  
  
idx = np.argsort(eigvals_U)[::-1]  
eigvals_U = eigvals_U[idx]  
U = U[:, idx]  
  
print("\n3. Lambda (AAT):\n", eigvals_U)
```

Wynik

3. Lambda (AAT):

```
[882.19305007 136.41976673 34.83692258 17.83495383 2.71530678]
```

3. U:

```
[[-0.29623956 -0.7044748 0.20578629 -0.16015708 -0.58988057]  
[-0.63526222 -0.33372816 -0.02940798 0.30917729 0.62338752]  
[-0.40730963 0.18439336 -0.40492725 -0.79539141 0.05902862]  
[-0.48273584 0.4303543 -0.31695392 0.4704922 -0.50984173]  
[-0.33128388 0.41609499 0.83208251 -0.15731721 0.0024371 ]]
```

4. Wypisanie/narysowanie macierzy wektorów własnych

$[U_1, U_2, \dots, U_n]$ oraz macierzy diagonalnej S takiej, że $S_{ii} = \sqrt{\lambda_i}$.

```
print("\n4. U:\n", U)
```

```
sigma = np.sqrt(np.clip(eigvals_U, 0, None))
```

```
print("\n4. S:\n", sigma)
```

Wynik

4. U:

```
[[-0.29623956 -0.7044748 0.20578629 -0.16015708 -0.58988057]  
[-0.63526222 -0.33372816 -0.02940798 0.30917729 0.62338752]  
[-0.40730963 0.18439336 -0.40492725 -0.79539141 0.05902862]  
[-0.48273584 0.4303543 -0.31695392 0.4704922 -0.50984173]  
[-0.33128388 0.41609499 0.83208251 -0.15731721 0.0024371 ]]
```

4. S:

```
[29.7017348 11.67988727 5.90228113 4.22314502 1.64781879]
```

5. Obliczenie macierzy V korzystając z własności $V = A^T \cdot U \cdot S^{-1}$

```
V = np.zeros((m, n))

for i in range(len(sigma)):
    if sigma[i] > 1e-10:
        V[:, i] = (A.T @ U[:, i]) / sigma[i]
```

6. Wypisanie/narysowanie macierzy $[V_1, V_2, \dots, V_m]^T$.

```
print("\n6. V^T:\n", V.T)
```

Wynik

```
6. V^T:
[[-0.37547136 -0.32688003 -0.51095593 -0.45959913 -0.31387864 -0.42584454]
 [ 0.2091492  -0.3572751   0.5169878  -0.02294322  0.26443972 -0.70062827]
 [-0.76389129  0.36255685  0.43130887 -0.24122657  0.2017232  -0.01061936]
 [-0.06303662  0.39226913 -0.51342821  0.34472849  0.54504544 -0.40327475]
 [-0.31110933 -0.00932658  0.11633455  0.72689778 -0.5532115  -0.23487639]]
```

7. Obliczenie i wypisanie macierzy $A^T \cdot A$ (wymiały $m \times m$).

```
ATA = A.T @ A
print("\n7. A^T A:\n", ATA)
```

Wynik

```
7. A^T A:
[[151.  88. 173. 157. 106. 122.]
 [ 88. 119. 124. 133.  84. 154.]
 [173. 124. 278. 199. 158. 146.]
 [157. 133. 199. 192. 127. 172.]
 [106.  84. 158. 127. 104.  89.]
 [122. 154. 146. 172.  89. 230.]]
```

8. Obliczenie wartości λ_i i wektorów własnych V_i macierzy $A^T \cdot A$.

```
eigvals_V, V2 = np.linalg.eigh(ATA)

idx = np.argsort(eigvals_V)[::-1]
eigvals_V = eigvals_V[idx]
V2 = V2[:, idx]

sigma2 = np.sqrt(np.clip(eigvals_V, 0, None))
```

9. Wypisanie/narysowanie macierzy $[V_1, V_2, \dots, V_m]^T$ i obliczenie i wypisanie macierzy diagonalnej S takiej, że $S_{ii} = \sqrt{\lambda_i}$.

```
print("\n9. V2:\n", V2)
print("\n9. S2:\n", sigma2)
```

Wynik

9. V2:

```
[[-0.37547136 -0.2091492  0.76389129  0.06303662 -0.31110933  0.36191895]
 [-0.32688003  0.3572751  -0.36255685 -0.39226913 -0.00932658  0.69288846]
 [-0.51095593 -0.5169878  -0.43130887  0.51342821  0.11633455  0.09207634]
 [-0.45959913  0.02294322  0.24122657 -0.34472849  0.72689778 -0.28780873]
 [-0.31387864 -0.26443972 -0.2017232  -0.54504544 -0.5532115  -0.43329215]
 [-0.42584454  0.70062827  0.01061936  0.40327475 -0.23487639 -0.33146077]]
```

9. S2:

```
[29.7017348  11.67988727  5.90228113  4.22314502  1.64781879  0.          ]
```

10. Obliczenie macierzy U korzystając z własności $U = A \cdot V \cdot S^{-1}$.

```
U2 = np.zeros((n, m))

for i in range(len(sigma2)):
    if sigma2[i] > 1e-10:
        U2[:, i] = (A @ V2[:, i]) / sigma2[i]
```

11. Wypisanie/narysowanie macierzy $[U_1, U_2, \dots, U_n]$.

```
print("\n10. U2:\n", U2)
```

Wynik

```
11. U2:
[[-0.29623956  0.7044748 -0.20578629  0.16015708 -0.58988057  0.          ]
 [-0.63526222  0.33372816  0.02940798 -0.30917729  0.62338752  0.          ]
 [-0.40730963 -0.18439336  0.40492725  0.79539141  0.05902862  0.          ]
 [-0.48273584 -0.4303543  0.31695392 -0.4704922 -0.50984173  0.          ]
 [-0.33128388 -0.41609499 -0.83208251  0.15731721  0.0024371  0.          ]]
```

12. Porównanie dekompozycji.

Aby porównać obie wersje dekompozycji zrekonstruowano macierz A za pomocą wyliczonych składowych oraz dokonano porównania z oryginalną wersją.

```

A_rec_1 = np.zeros((n, m))
A_rec_2 = np.zeros((n, m))

r = min(n, m)

for i in range(r):
    A_rec_1 += sigma[i] * np.outer(U[:, i], V[:, i])
    A_rec_2 += sigma2[i] * np.outer(U2[:, i], V2[:, i])

print("\nA_rec_1: ", A_rec_1)
print("\nA_rec_2: ", A_rec_2)

print("\n12. Błąd AAT:", np.linalg.norm(A - A_rec_1))
print("12. Błąd ATA:", np.linalg.norm(A - A_rec_2))
print("12. Różnica metod:", np.linalg.norm(A_rec_1 - A_rec_2))

```

Wynik

```

A_rec_1: [[ 1.  6.  1.  3.  1. 10.]
 [ 6.  8.  7. 10.  5. 10.]
 [ 7.  1.  8.  5.  2.  5.]
 [ 8.  3.  8.  7.  7.  2.]
 [ 1.  3. 10.  3.  5.  1.]]

```

```

A_rec_2: [[ 1.  6.  1.  3.  1. 10.]
 [ 6.  8.  7. 10.  5. 10.]
 [ 7.  1.  8.  5.  2.  5.]
 [ 8.  3.  8.  7.  7.  2.]
 [ 1.  3. 10.  3.  5.  1.]]

```

12. Błąd AAT: 2.5019215181393538e-14

12. Błąd ATA: 4.216860310682953e-14

12. Różnica metod: 3.8270339404765685e-14

Zrekonstruowane macierze są niemalże identyczne w porównaniu z oryginalną macierzą A . Wyliczone błędy są nieznaczne, co wskazuje na poprawność napisanego programu.

13. Wyliczenie $\dim R(A)$ oraz $\dim N(A)$

```
rank = np.linalg.matrix_rank(A)
null_dim = m - rank

print("\n13. dim R(A):", rank)
print("13. dim N(A):", null_dim)
```

Wynik

```
13. dim R(A): 5
13. dim N(A): 1
```

Interpretacja wyników:

- $\dim R(A) = 5$ oznacza, że macierz opisuje wszystkie wymiary przestrzeni,
- $\dim N(A) = 1$ oznacza, że macierz redukuje przestrzeń wejściową o jeden wymiar.

Wnioski

- Przeprowadzone obliczenia potwierdzają, że rozkład SVD macierzy A może być wyznaczony na dwa równoważne sposoby - poprzez analizę spektralną macierzy $A \cdot A^T$, oraz $A^T \cdot A$. Obie metody prowadzą do tych samych wartości singularnych (z dokładnością numeryczną).
- Wektory własne macierzy $A \cdot A^T$ tworzą ortonormalną bazę przestrzeni kolumnowej macierzy A (macierz U), natomiast wektory własne macierzy $A^T \cdot A$ tworzą ortonormalną bazę przestrzeni wierszowej (macierz V).
- Wartości singularne S_{ii} są pierwiastkami z wartości własnych obu macierzy symetrycznych $A \cdot A^T$ i $A^T \cdot A$, co potwierdza ich równoważność spektralną w kontekście SVD.
- Rekonstrukcja macierzy A w postaci $A = U \cdot S \cdot V^T$ wykazuje bardzo mały błąd numeryczny (rzędu błędów maszynowych), co potwierdza poprawność implementacji oraz stabilność dekompozycji.
- Rząd macierzy A odpowiada liczbie niezerowych wartości singularnych, co umożliwia bezpośrednią interpretację wymiaru przestrzeni obrazu $R(A)$.
- Wymiar jądra $N(A)$ wynika z zależności $\dim N(A) = m - \text{rank}(A)$, co jest zgodne z twierdzeniem o rzędzie i jądrze odwzorowania liniowego.
- Numeryczne porównanie obu podejść (przez $A \cdot A^T$ i $A^T \cdot A$) wskazuje, że mimo potencjalnych różnic w znakach wektorów własnych, struktura SVD pozostaje niezmienna.